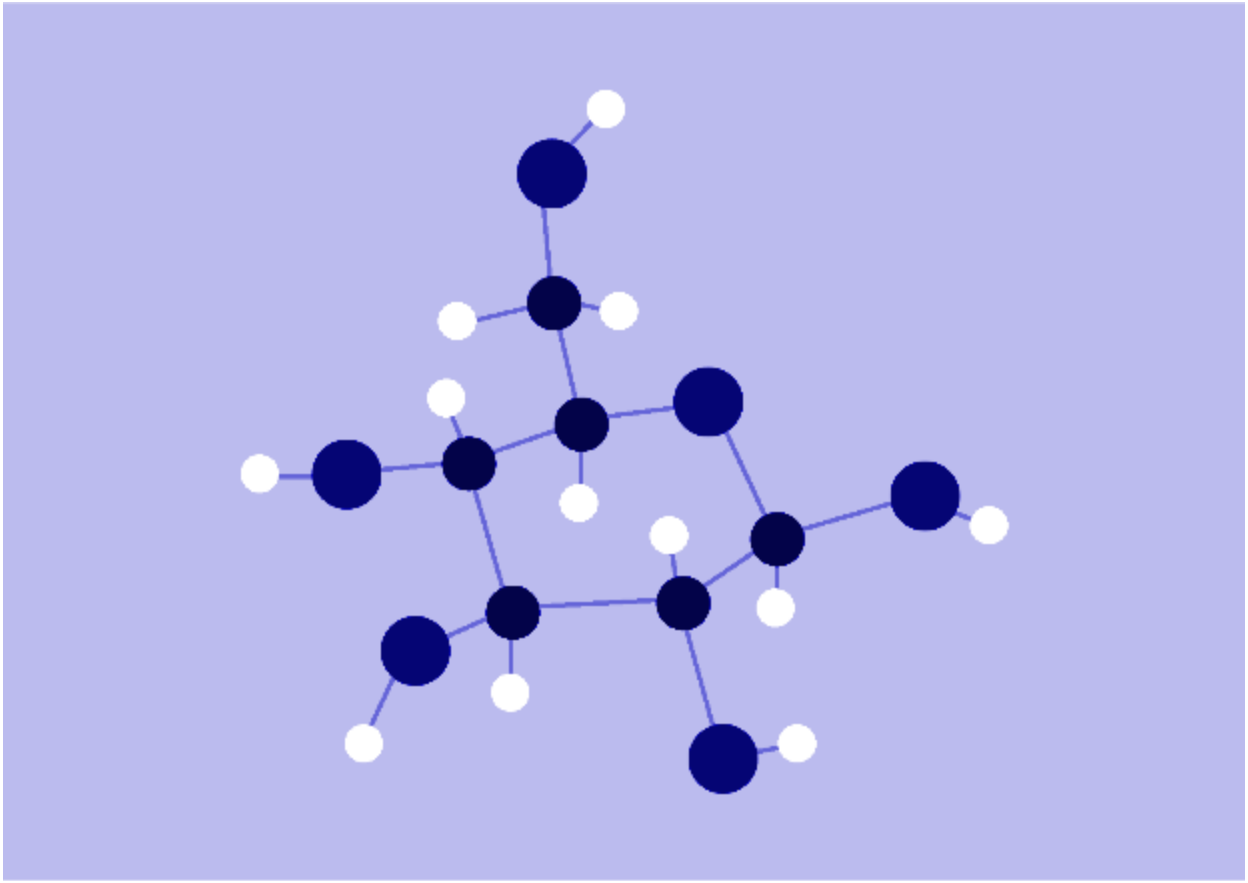
Design-Entwurf

ChronoScope - Online Aufgaben-Planungssystem/Selbst-Managementsystem



NI0 - Name Ideas 0

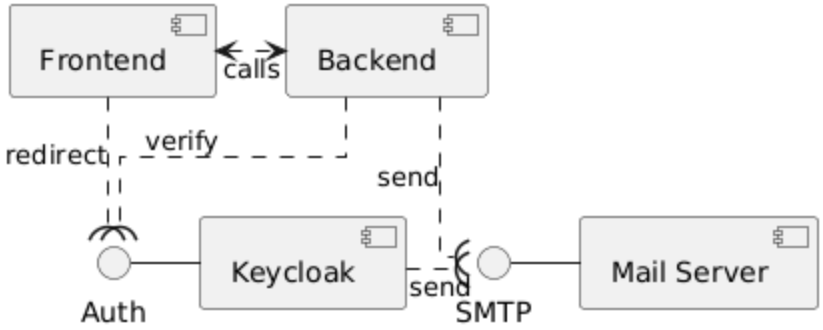
Stand 12.04.2026

Autoren: Felix Weglehner, Konstantin Fastovski, Elias Kerlin

Prüfleser: Paul Berndt

Einführung 2
Authentifizierung 2
Backend 5
Frontend 10
Algorithmus 13
Schnittstelle 17

Einführung



Das Gesamtsystem besteht aus Frontend, Backend, Keycloak und dem Mailserver.

Das Frontend tauscht hauptsächlich Daten mit dem Backend aus, um die für die Anzeige benötigten Informationen abzurufen. Die Kommunikation erfolgt über eine REST-API, die durch eine OpenAPI-Spezifikation definiert ist. Für die Authentifizierung leitet das Frontend die Benutzer zur Keycloak-Oberfläche weiter. Auch das Backend kommuniziert mit Keycloak, um vom Frontend übermittelte Tokens zu überprüfen. Dabei wird die Signatur im JWT mithilfe eines von Keycloak bereitgestellten Public Keys validiert.

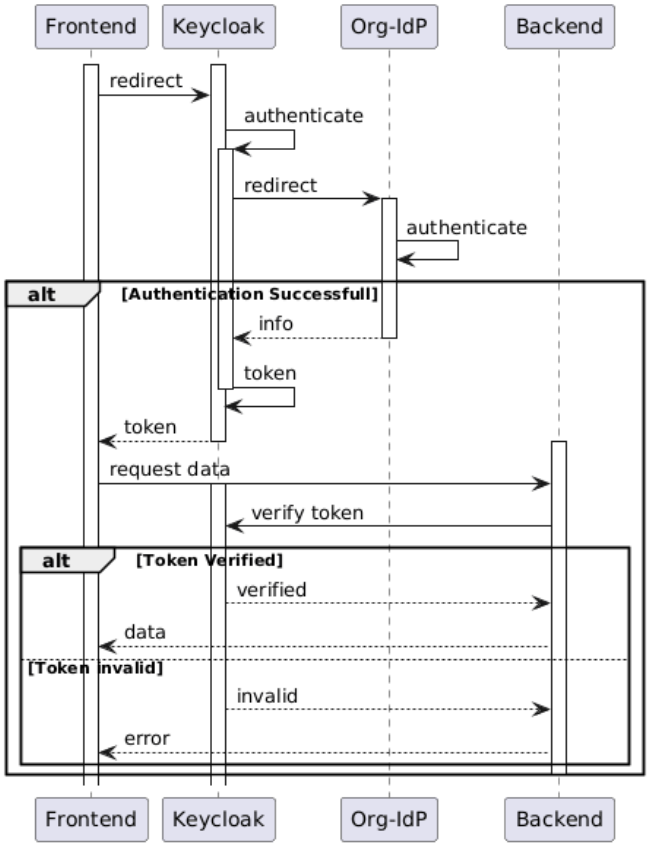
Sowohl das Backend als auch Keycloak müssen E-Mails versenden können, beispielsweise für Passwort-Resets oder die Verknüpfung von Accounts. Dazu stellen sie Anfragen an den Mailserver. Der Mailserver ist ein SMTP-Server, der E-Mails unter der Domain ni0.team verschicken kann.

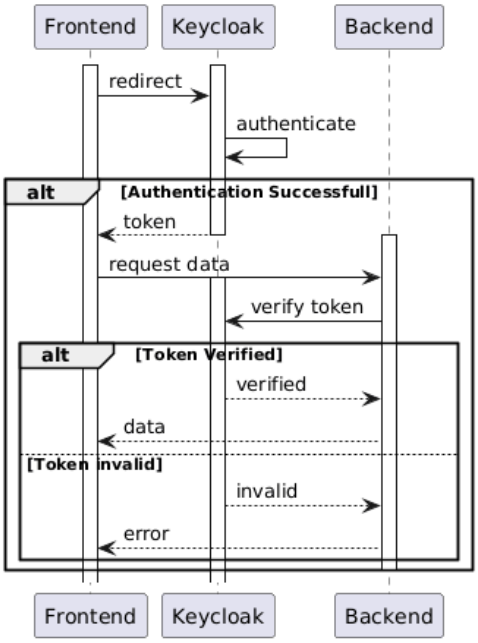
Authentifizierung

Keycloak dient als zentraler Identity Provider. Die Authentifizierung über Keycloak kann auf zwei unterschiedliche Arten erfolgen.

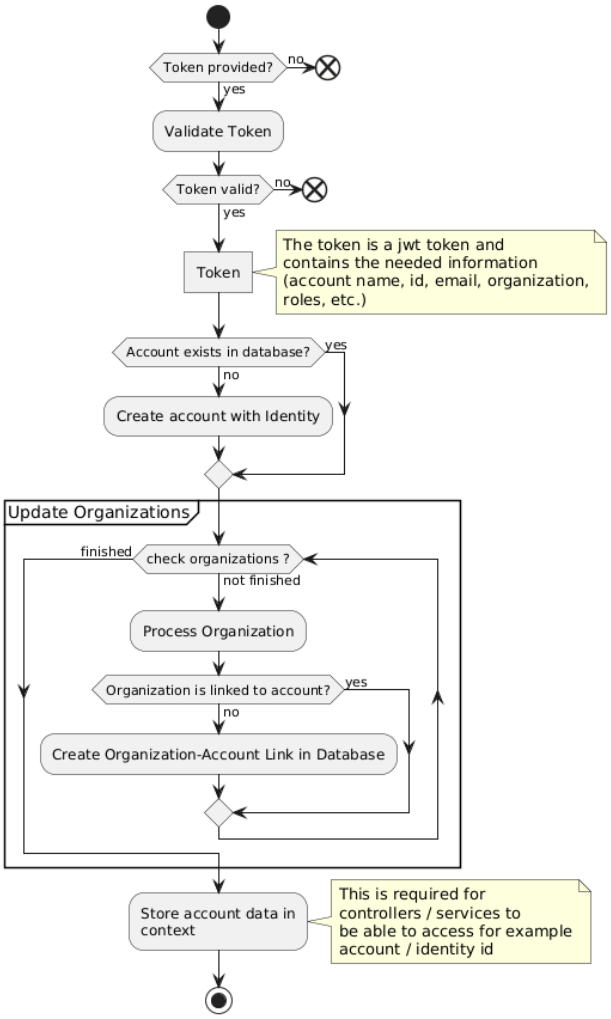
Bei einer direkten Anmeldung bei Keycloak können sich private Nutzer registrieren und die Anwendung ohne Zugehörigkeit zu einer Organisation nutzen. Wird eine E-Mail-Adresse eingegeben, die von einer Organisation verwaltet wird, leitet Keycloak die Authentifizierung über den Identity Provider dieser Organisation weiter.

Die folgenden Diagramme veranschaulichen beide Möglichkeiten:

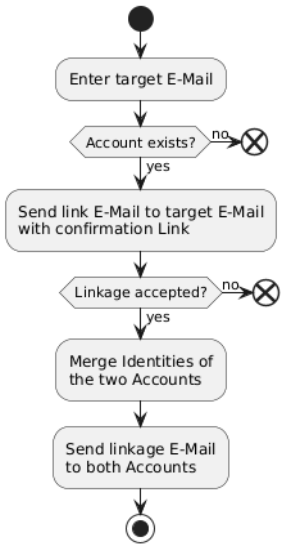




Sowohl das Frontend als auch das Backend haben Zugriff auf den JWT-Token. Aus dem Token können die Systeme wertvolle Informationen über den angemeldeten Nutzer auslesen, etwa Name, E-Mail-Adresse, Gruppenmitgliedschaften, zugeordnete Organisationen und weitere Daten.



Im Backend wird der Token über die Spring-Security-Chain ausgelesen und validiert. Anschließend werden die benötigten Informationen in der Datenbank sowie im Context gespeichert. Das nebenstehende Diagramm veranschaulicht diesen Vorgang.

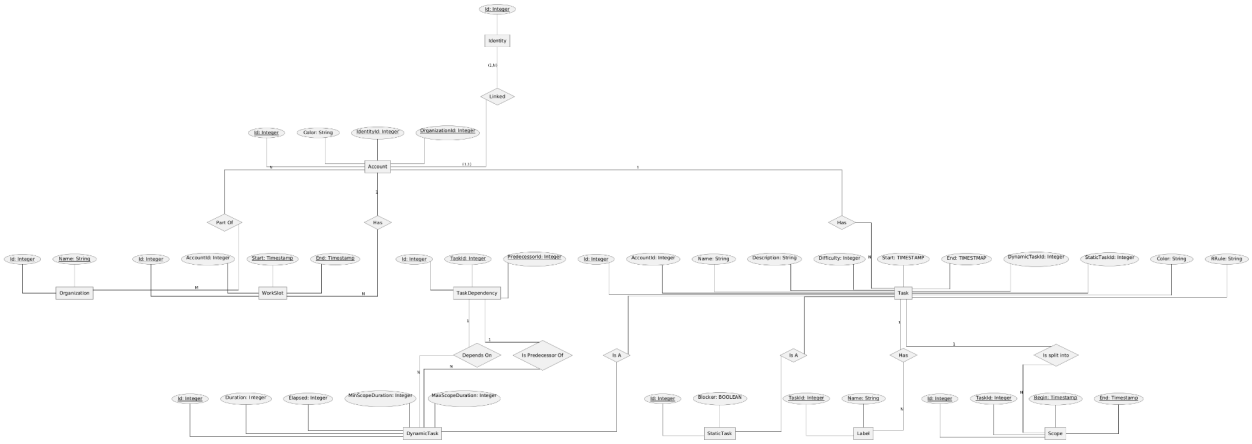


Wenn ein Nutzer Accounts mehrerer Organisationen besitzt, soll er diese verbinden können, um organisationsübergreifend arbeiten zu können. Das folgende Diagramm veranschaulicht den Verknüpfungsvorgang. Dafür ist eine Kommunikation zwischen Frontend und Backend erforderlich, die in diesem Diagramm nicht dargestellt wird.

Um die Verbindung mehrerer Accounts zu speichern, werden die Accounts über eine Identität verknüpft, die eine Person repräsentiert. Im Folgenden wird die Modellierung der Datenbank und des Backends in Bezug auf diese Daten beschrieben.

Backend

Das folgende Schaubild zeigt die Modellierung der Objekte im Backend.



Das Domänenmodell gliedert sich in mehrere zusammenhängende Bereiche, die im Folgenden erläutert werden:

Nutzer- und Organisationsstruktur: Eine `Identity` repräsentiert eine reale Person und dient als gemeinsamer Anker für mehrere Accounts derselben Person. Ein `Account` ist die nutzerspezifische Ausprägung innerhalb einer Organisation und enthält beispielsweise Attribute wie eine Farbe zur Anzeige im Kalender. Die Verknüpfung zwischen `Account` und `Organization` ist eine N-M-Beziehung: Ein Benutzer kann Mitglied mehrerer Organisationen sein, und eine Organisation kann viele Nutzer haben. Über die übergeordnete `Identity` können Accounts verschiedener Organisationen einer einzelnen Person zugeordnet werden, ohne die organisatorische Trennung aufzuheben. Dies ermöglicht das in der Authentifizierungs-Sektion beschriebene Account-Linkage-Verfahren.

Arbeitszeiten: `WorkSlots` definieren die Zeitfenster, in denen ein Nutzer planbare Aufgaben bearbeiten kann – zum Beispiel werktags von 7–12 und 13–16 Uhr. Der Planungsalgorithmus verteilt dynamische Tasks ausschließlich innerhalb dieser Zeitfenster.

Aufgaben und Typen: Eine `Task` ist die zentrale Entität des Systems. Jede Aufgabe besitzt eine Verknüpfung zum zugehörigen `Account` sowie optionale Fremdschlüssel

zu `DynamicTask` und `StaticTask`, über die das Aufgabentyp-Konzept durch Komposition realisiert wird:

- Eine statische Aufgabe (Termin, Blocker) erhält einen Eintrag in `StaticTask` – das `blocker`-Flag gibt dabei an, ob die Aufgabe als harter Blocker für den Planungsalgorithmus gilt.
- Eine planbare Aufgabe erhält einen Eintrag in `DynamicTask`, der alle für den Algorithmus notwendigen Attribute enthält: `difficulty` (Schwierigkeit, fließt in die Heuristik ein), `duration` (geschätzter Gesamtaufwand), `elapsed` (bereits abgearbeitete Zeit) sowie `start` und `end` (Frist-Fenster, in dem die Aufgabe eingeplant werden muss).

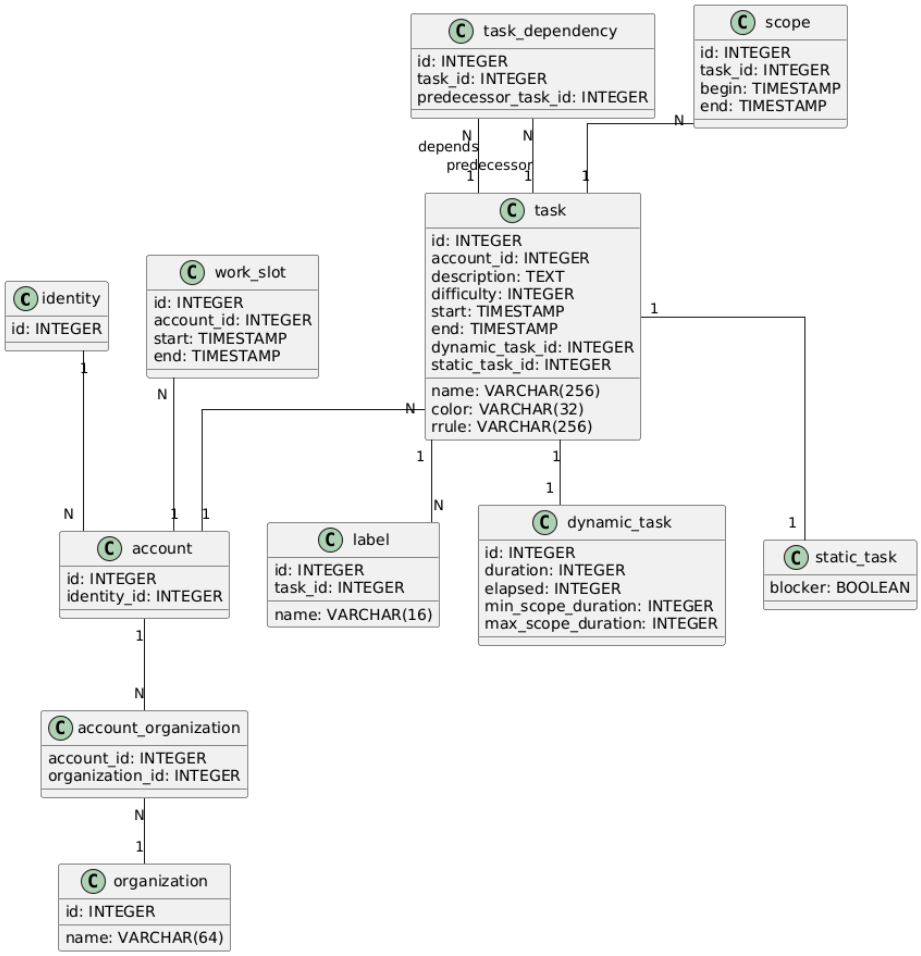
Das Feld `rrule` im `Task` enthält einen optionalen RRULE-String nach RFC 5545 und ermöglicht wiederkehrende Aufgaben nach einem standardisierten Wiederholungsmuster.

Abhängigkeiten und Scopes: `TaskDependency` modelliert die Abhängigkeiten zwischen Aufgaben: Eine Abhängigkeit zeigt an, dass eine Aufgabe erst begonnen werden darf, wenn ihre Vorgänger abgeschlossen sind. Diese Beziehungen bilden den gerichteten azyklischen Graphen (DAG), den der Planungsalgorithmus topologisch sortiert.

`Scopes` sind die vom Algorithmus erzeugten konkreten Arbeitsblöcke, in die eine planbare Aufgabe aufgeteilt wird. Eine `Task` kann in mehrere zeitlich getrennte `Scopes` aufgespalten werden, wenn ein einzelner `WorkSlot` nicht ausreicht, um die gesamte `duration` unterzubringen.

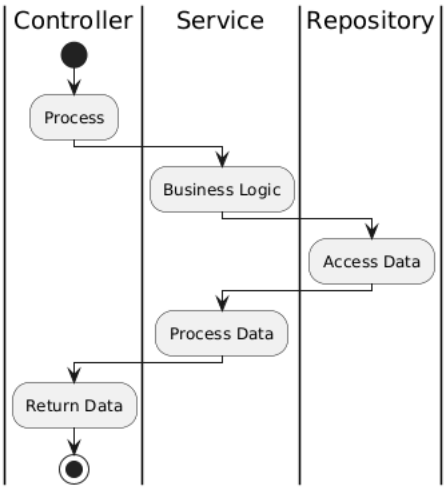
Labels: Jede Aufgabe kann mit beliebig vielen `Labels` versehen werden, die aus einem Namen bestehen und zur Kategorisierung und Filterung von Aufgaben im Frontend dienen.

Die Repräsentation in der Datenbank ist dem folgenden ER-Modell sehr ähnlich:



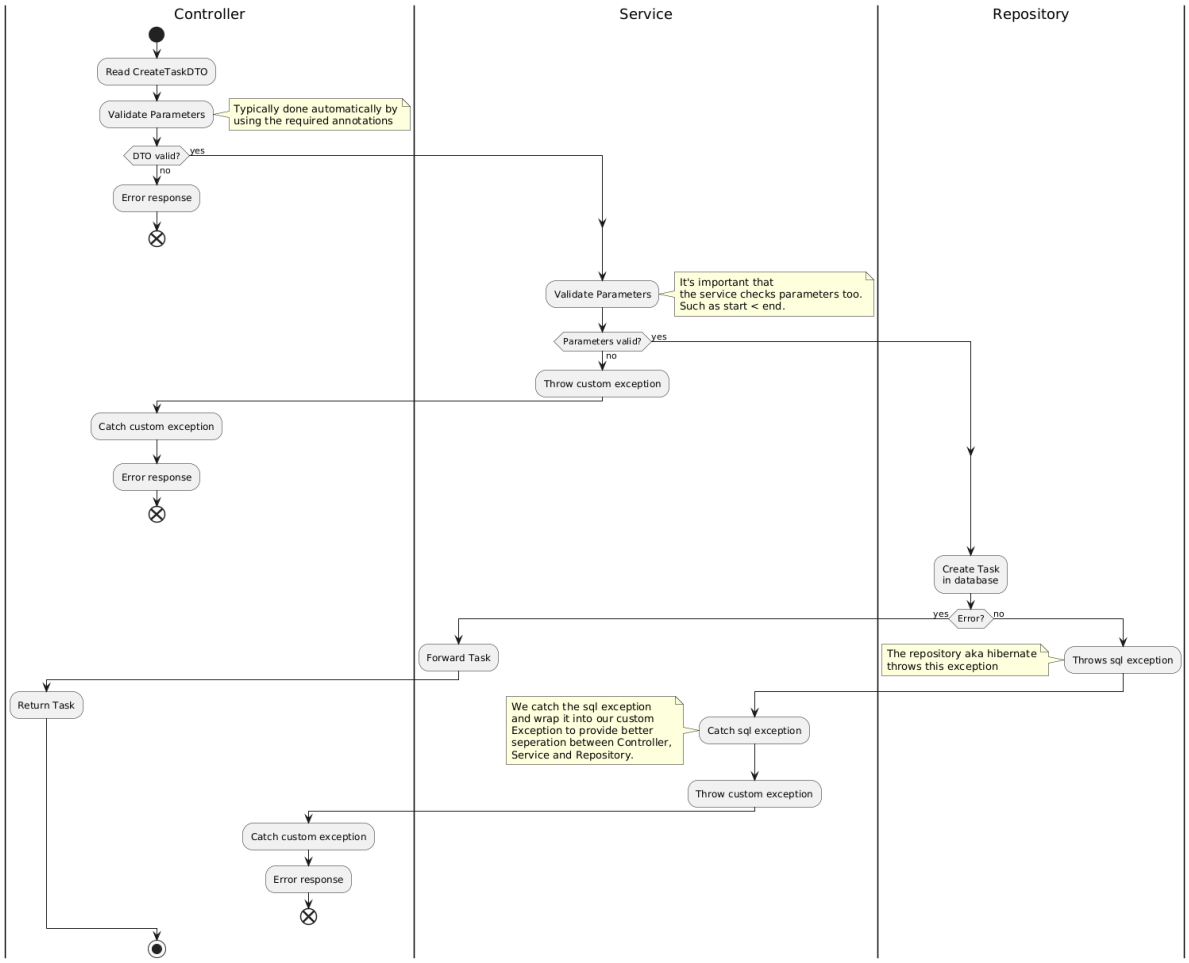
Zu beachten ist, dass die N-M-Beziehung zwischen Accounts und Organisationen durch eine `account_organization`-Tabelle abgebildet wird. Die Vererbung der Tasks wurde über Komposition umgesetzt. Zusätzlich zur `task`-Tabelle existiert für dynamische Tasks ein Eintrag in der `dynamic_task`-Tabelle und für statische Tasks ein Eintrag in der `static_task`-Tabelle. Diese Einträge werden in der `task`-Tabelle referenziert und können gegebenenfalls null sein.

Ein Task enthält ein optionales Feld `RRULE`, das einen RRULE-String nach RFC 5545 enthält. Dieser Standard definiert wiederkehrende Termine, für die sowohl im Frontend als auch im Backend etablierte Bibliotheken existieren. Das `blocker`-Attribut in `static_task` gibt an, ob der `Task` als Termin oder Blocker angezeigt wird und kann gegebenenfalls die Zeiterfassung beeinflussen. In `dynamic_task` sind alle Attribute gespeichert, die für den Algorithmus notwendig sind.



Das Backend ist nach der typischen Struktur einer Java-Spring-Anwendung aufgebaut. Es existieren mehrere Controller, Services und Repositories, die als Schichten dienen und miteinander kommunizieren.

Beim Erstellen eines Tasks werden die drei beschriebenen Ebenen in der folgenden Reihenfolge durchlaufen:

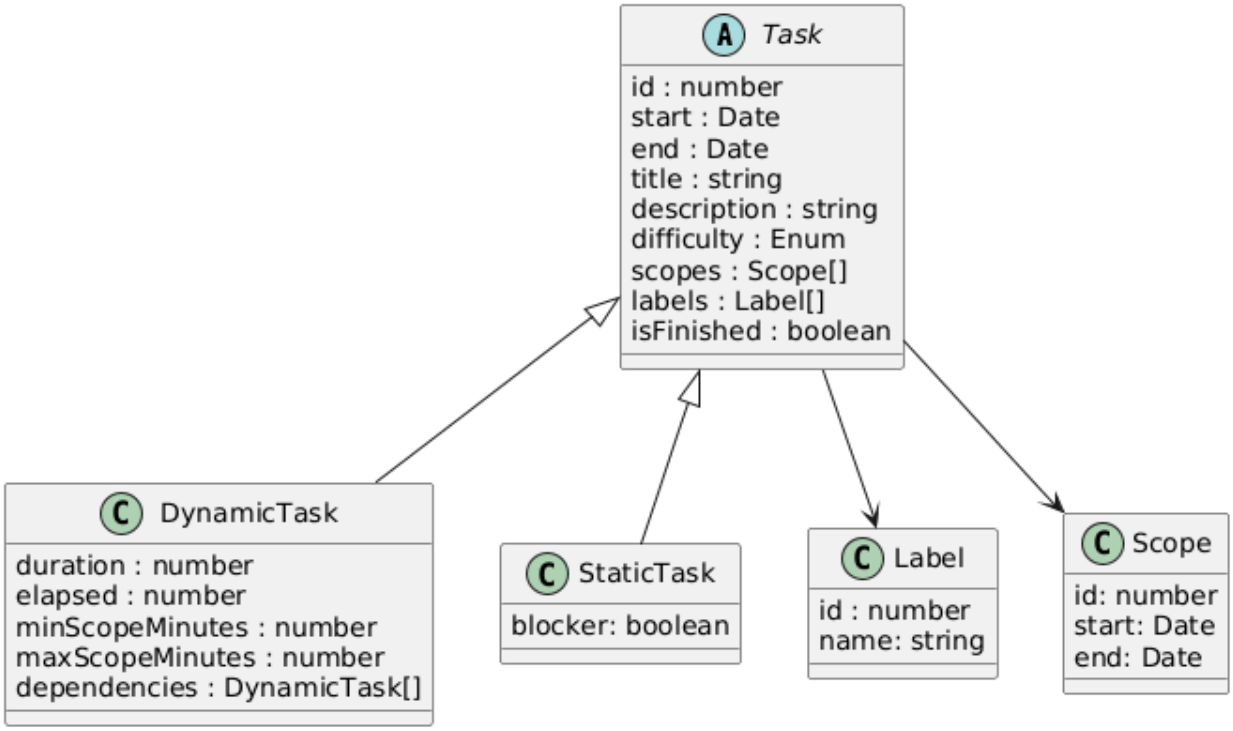


Es muss auf eine strikte Trennung der Schichten geachtet werden: Jede Schicht darf ausschließlich ihre eigenen Verantwortlichkeiten enthalten. Fehler, die von der Datenbank zurückgegeben werden, werden in der Service-Schicht verarbeitet, und es werden geeignete Exceptions geworfen. In der Service-Schicht sollten keine DTOs oder andere HTTP-spezifischen Elemente verwendet werden.

Andere Anwendungsfälle, wie das Löschen oder Bearbeiten von Tasks sowie das Ändern von Arbeitszeiten oder das Laden von Tasks, folgen demselben Schema und werden daher hier nicht in separaten Diagrammen dargestellt (Controller → Service → Repository → Service → Controller).

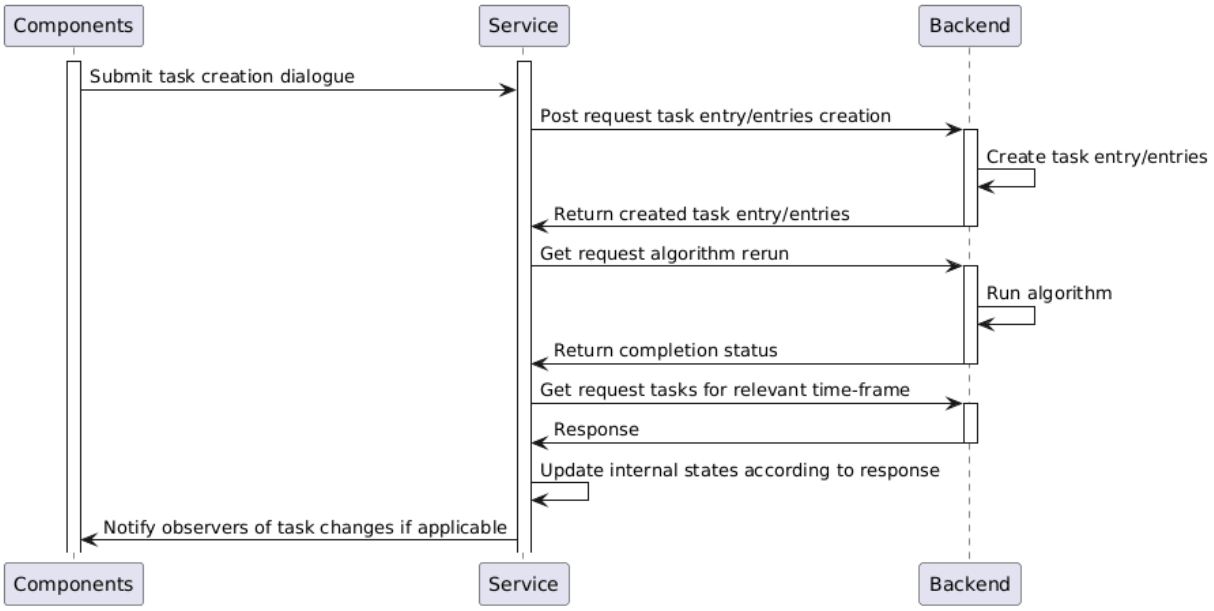
Frontend

Dieses UML-Klassendiagramm zeigt die Klassen, mit denen Aufgaben und Termine im internen Zustand des Frontends gespeichert werden.



Einige vom Backend modellierte Attribute sind im Frontend nicht enthalten, da sie für den Benutzer nicht relevant sind. Im Frontend sind lediglich die Attribute verfügbar, die der Benutzer auf der Anzeige sieht oder beim Erstellen einer Aufgabe benötigt.

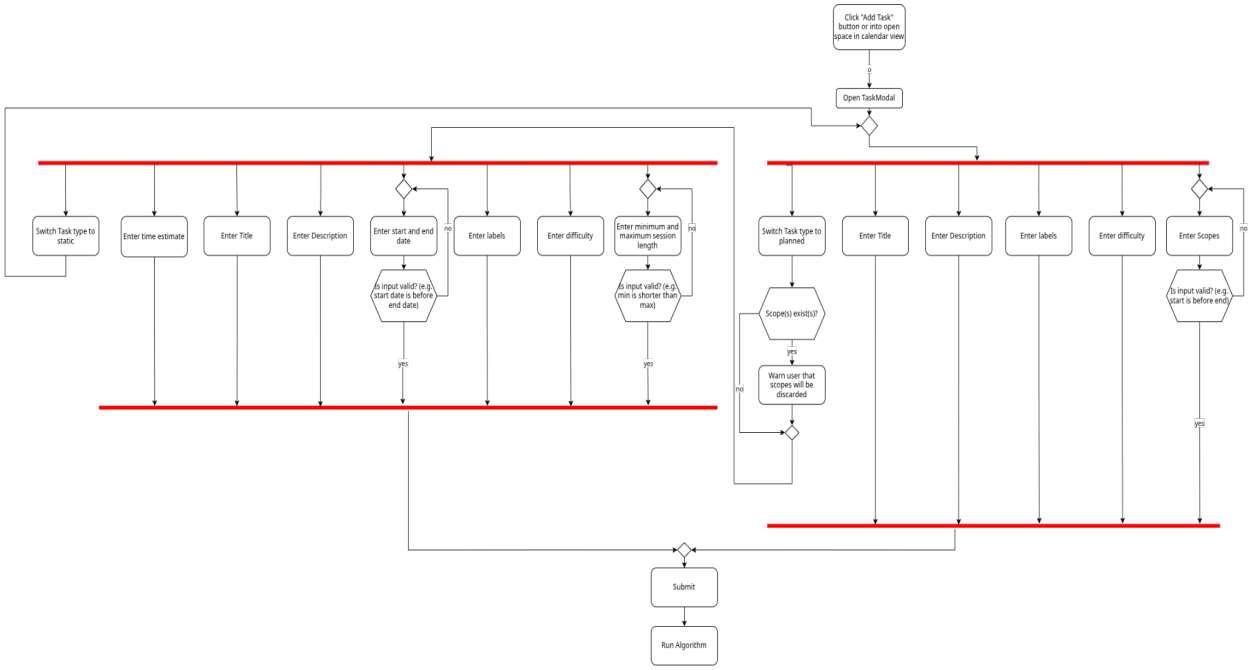
Das folgende Sequenzdiagramm zeigt den Ablauf der Aufgabenerstellung zwischen den Frontend-Komponenten, dem Task-Service und dem Backend.



Nach den Angular-Best-Practices wird das Observer-Pattern verwendet, um den Kalender und alle relevanten Komponenten über Änderungen im internen Zustand des Task-Service zu informieren. Dieser Service nutzt die REST-API, um das Backend anzuweisen, eine neue Aufgabe anzulegen, sobald der Benutzer eine Aufgabe erstellt.

Erst nach Erhalt einer erfolgreichen Antwort fordert der Service eine Neuplanung der Aufgaben durch den Algorithmus an. Nach erfolgreichem Abschluss dieser Neuplanung werden die Aufgaben für einen begrenzten Zeitraum abgefragt, damit sie im Frontend angezeigt werden können. Dieser Zeitraum richtet sich nach der aktuell vom Benutzer gewählten Ansicht. Ein Abruf aller Aufgaben nach jeder Neuplanung – einschließlich vergangener oder weit in der Zukunft liegender Aufgaben – ist ausgeschlossen.

Um die Aufgabenerstellung auch aus der Sicht des Benutzers zu veranschaulichen, liegt folgendes Aktivitätsdiagramm für die Formulare vor, die der Benutzer ausfüllt:



Durch das Drücken des „Add Task"-Buttons oder einen Klick auf eine freie Stelle im Kalender wird das Eingabefeld für eine neue Aufgabe geöffnet. Standardmäßig ist eine `StaticTask` ausgewählt. Dort kann der Benutzer in beliebiger Reihenfolge Titel, Beschreibung, Labels, Schwierigkeit und `Scopes` (Arbeitsblöcke) angeben.

Neben einer allgemeinen Validierung der Eingabedaten (z. B. Eingabelänge), die hier nicht explizit beschrieben wird, prüft das Frontend für die `Scopes` einige grundlegende Fehler, bevor die Aufgabe nach vollständiger Eingabe angelegt und der Algorithmus ausgeführt werden kann.

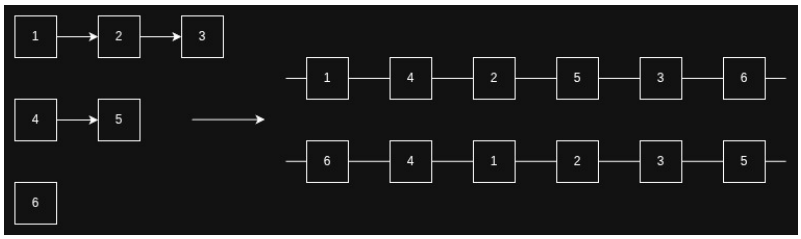
Vor der Anlage der Aufgabe besteht jederzeit die Möglichkeit, den Aufgabentyp auf „Planned" zu ändern. Sollten bereits `Scopes` existieren, wird der Benutzer darauf hingewiesen, dass diese Aktion die bestehenden `Scopes` löscht. Nach Bestätigung wird der Aufgabentyp entsprechend geändert.

Bei `PlannedTasks` kann der Benutzer Titel, Beschreibung, Zeiteinschätzung, Start- und Enddatum, Labels, Schwierigkeit sowie die minimale und maximale Länge einer Arbeitsperiode (`Scopes`) in beliebiger Reihenfolge eingeben. Sowohl die Dauer der Arbeitsperiode als auch das Start- und Enddatum werden auf Plausibilität überprüft, bevor die Aufgabe endgültig angelegt werden kann. Auch bei `PlannedTasks` kann der Benutzer den Aufgabentyp jederzeit wieder ändern.

Algorithmus

Im folgenden Abschnitt wird die Funktionsweise des Planungsalgorithmus beschrieben. Die Darstellung erfolgt zunächst in Worten, um die Verständlichkeit zu erhöhen. Ein Aktivitätsdiagramm am Ende ergänzt die Beschreibung.

Bei der Planung muss der Algorithmus Abhängigkeiten zwischen Tasks berücksichtigen: Task A muss vor Task B abgeschlossen werden, wenn B von A abhängig ist. Dafür werden alle Abhängigkeiten mithilfe eines topologischen Sortierverfahrens geordnet. Die Tasks bilden einen Directed Acyclic Graph (DAG), und zum Sortieren wird eine angepasste Variante des Depth-First-Search-Algorithmus verwendet.



Dabei ist zu beachten, dass es nicht die eine „richtige“ Reihenfolge gibt. Im Beispiel ist Task 3 abhängig von Task 2, Task 2 abhängig von Task 1 und Task 5 abhängig von Task 4. Der erste Task könnte daher 1, 4 oder 6 sein. Der Algorithmus plant die Aufgaben schließlich in einer dieser möglichen Reihenfolgen ein.

Welche Reihenfolge gewählt wird, entscheidet eine Heuristik. Vor der Auswahl des nächsten zu planenden Tasks wird für jeden möglichen Task ein Heuristikwert berechnet. Dieser Wert gibt an, wie sinnvoll es erscheint, den Task als nächstes einzuplanen. Anschließend wird der Task mit dem höchsten (bzw. niedrigsten) Wert ausgewählt.

Ein Bestandteil der Heuristik sind kritische Pfade. Kritische Pfade bestehen aus einzelnen Tasks oder durch Abhängigkeiten verbundenen Ketten, die nur wenig Flexibilität in der Planung erlauben. Wenn Start- und Enddatum einer Aufgabe oder mehrerer Aufgaben den Planungsspielraum stark einschränken, sollten diese Aufgaben bevorzugt eingeplant werden. Zur Berechnung der kritischen Pfade wird ein CPM-Algorithmus (Critical Path Method) eingesetzt, der für jeden Task vier Zeitpunkte ermittelt.

- **Earliest Start (ES):** Zeitpunkt, zu dem der Task frühestmöglich beginnen kann
- **Earliest Finish (EF):** Zeitpunkt, zu dem der Task frühestmöglich beendet werden kann
- **Latest Start (LS):** Zeitpunkt, zu dem der Task spätestens beginnen darf
- **Latest Finish (LF):** Zeitpunkt, zu dem der Task spätestens beendet sein muss

Der **Slack** (Planungsflexibilität) ergibt sich aus der Differenz zwischen Latest Start und Earliest Start. Aufgaben mit geringer Planungsflexibilität sollten frühzeitig eingeplant werden.

In die Heuristik fließt außerdem die Schwierigkeit der Aufgaben ein. Wenn die zuletzt drei geplanten Aufgaben sehr komplex waren, sollte zunächst eine Aufgabe mit niedrigerer Komplexität ausgewählt werden. Hierfür kann die Komplexität des möglichen nächsten Tasks mit dem Durchschnitt der letzten zwei Tasks verglichen werden.

`dc = abs(c - (c1+c2)/2)` (Je größer der Unterschied, desto höher die Priorität)

Eine mögliche abstrakte Darstellung der Heuristik lautet somit:

`h = w1 * 1 / slack  + w2 * dc` (w1 und w2 sind Gewichtungen, die konfiguriert werden müssen). Auf Grundlage dieser Heuristik wird anschließend der nächste Task ausgewählt und eingeplant.
Der Algorithmus liefert gute Ergebnisse, garantiert jedoch kein optimales Ergebnis. Es ist in polynomieller Zeit nicht möglich, eine vorhandene Lösung sicher zu finden. Durch Backtracking können jedoch zusätzliche Pfade geprüft werden. Diese Pfade sind laut Heuristik möglicherweise nicht optimal, können aber dennoch zu einer Lösung führen.

Im oben genannten Beispiel ist der Start bei Task 1 laut Heuristik vermutlich am sinnvollsten, es spricht jedoch nichts dagegen, mit Task 4 zu beginnen. Alle diese Pfade lassen sich bildlich als Baum darstellen. Erkennt der Algorithmus während der Planung, dass eine Deadline nicht eingehalten werden kann, springt er zu der Auswahl des entsprechenden Tasks (ein Knoten im Baum) zurück, wählt einen anderen Task (Pfad)

aus und prüft erneut.

Würde der Algorithmus alle Pfade prüfen, wäre garantiert, dass eine Lösung gefunden wird, falls eine existiert. In der Praxis würde dies jedoch zu viel Zeit in Anspruch nehmen. Stattdessen kann ein Kompromiss gewählt werden, indem beispielsweise nur die laut Heuristik 10.000 vielversprechendsten Pfade geprüft werden.

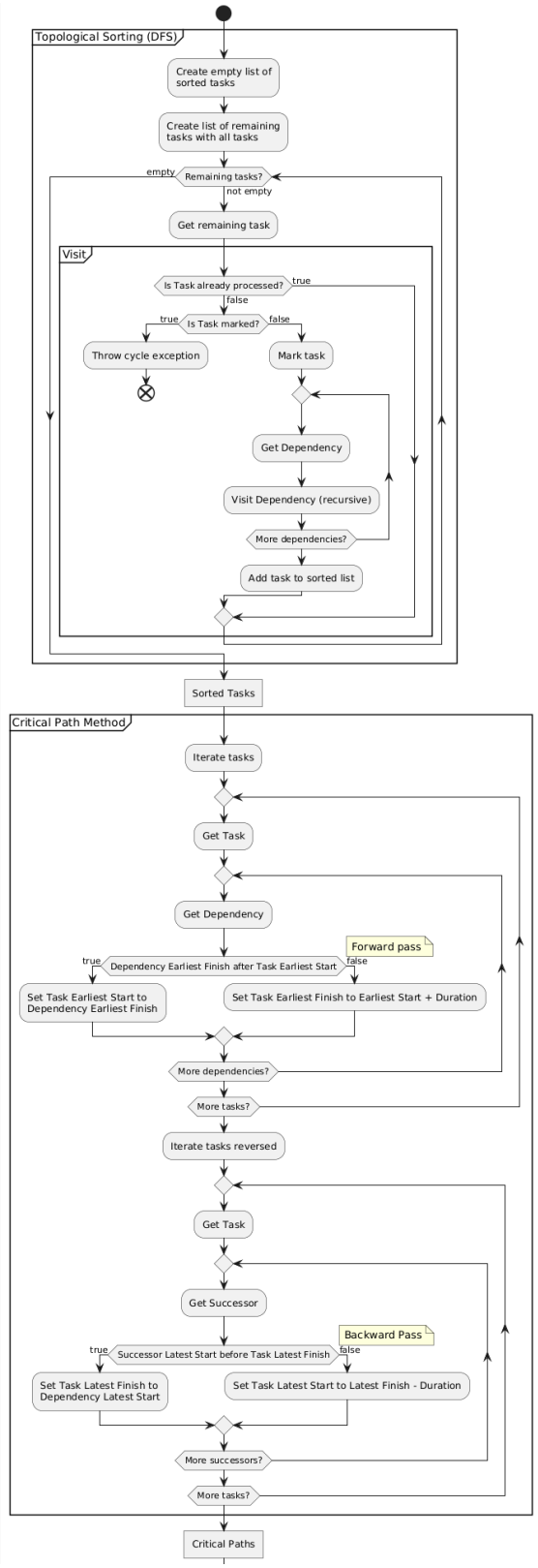
Durch die Möglichkeit, einen Pfad sofort abzubrechen, sobald eine Deadline nicht mehr eingehalten werden kann, lassen sich im besten Fall ganze Teilbäume ausschließen. Auf diese Weise entsteht eine Art **Branch-and-Bound-Algorithmus**.

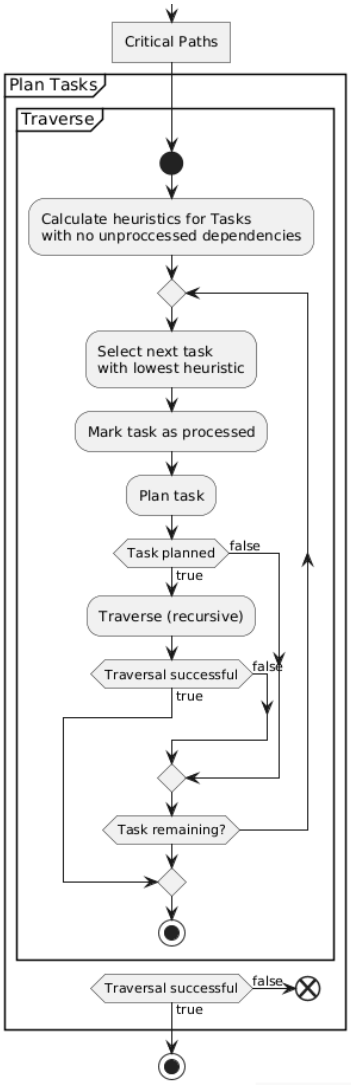
Die eigentliche Einplanung der Tasks erfolgt durch die Zuordnung zu den Arbeitszeiten des Benutzers. Diese Arbeitszeiten sind Zeitbereiche, beispielsweise von 7 – 12 Uhr und 13 – 16 Uhr, in denen Aufgaben eingeplant werden sollen.

Die Tasks werden dabei „stumpf“ auf die Arbeitszeiten verteilt. Sollte der verbleibende Zeitraum in einem Abschnitt nicht ausreichen, können Tasks in mehrere Teile (Scopes) aufgeteilt werden. In diesem Schritt lassen sich zusätzliche Anforderungen implementieren, etwa „zwischen jedem Scope 5 Minuten Pause“.

Wenn der Algorithmus die für jeden Task verbleibende einzuplanende Zeit im Rekursionsbaum weitergibt, entsteht zudem die Möglichkeit, einen anderen Task „zwischenzuschieben“, obwohl der aktuelle Task noch nicht vollständig eingeplant ist. Dies kann genutzt werden, um mehr Abwechslung im Arbeitsalltag zu schaffen.

Im folgenden umfassenden Aktivitätsdiagramm wird die Funktionsweise der einzelnen Komponenten im Detail dargestellt:


Topological Sorting (DFS)
Create empty list of sorted tasks
Create list of remaining tasks with all tasks
Remaining tasks?
empty
not empty
Get remaining task
Visit
Is Task already processed?
true
false
Is Task marked?
true
false
Throw cycle exception
Mark task
Get Dependency
Visit Dependency (recursive)
More dependencies?
Add task to sorted list
Sorted Tasks
Critical Path Method
Iterate tasks
Get Task
Get Dependency
Forward pass
Dependency Earliest Finish after Task Earliest Start
true
false
Set Task Earliest Start to Dependency Earliest Finish
Set Task Earliest Finish to Earliest Start + Duration
More dependencies?
More tasks?
Iterate tasks reversed
Get Task
Get Successor
Backward Pass
Successor Latest Start before Task Latest Finish
true
false
Set Task Latest Finish to Dependency Latest Start
Set Task Latest Start to Latest Finish - Duration
More successors?
More tasks?
Critical Paths


Critical Paths
Plan Tasks
Traverse
Calculate heuristics for Tasks with no unproccessed dependencies
Select next task with lowest heuristic
Mark task as processed
Plan task
Task planned
false
true
Traverse (recursive)
Traversal successful
false
true
Task remaining?
Traversal successful
false
true

Schnittstelle

Die Schnittstelle zwischen Frontend und Backend wird durch eine OpenAPI-Spezifikation definiert:

https://raw.githubusercontent.com/NI0-Name-Ideas-0/ChronoScope-models/refs/heads/main/openapi.yaml
Die API definiert wichtige Endpunkte, gegen die Backend und Frontend implementiert werden. So wird eine einheitliche Funktionsweise gewährleistet.

Die API stellt verschiedene Endpunkte für die Verwaltung von Nutzeridentitäten bereit:

- `/identity`**:** Ruft die eigene Identität ab, die aus dem Access Token gelesen wird. Eine Identität kann mehrere Accounts besitzen.
- `/identity/accounts`**:** Listet alle Accounts einer Identität auf. Accounts können miteinander verknüpft werden, indem zunächst eine Verknüpfungsanfrage per E-Mail ausgelöst wird und anschließend ein Token zur Bestätigung genutzt wird.
- `/accounts/{accountid}`**:** Liefert detaillierte Informationen zu einem bestimmten `Account` sowie den zugehörigen Organisationen. Organisationen sind Gruppen, denen ein `Account` angehören kann.

Ein zentraler Bestandteil der API ist die Verwaltung von Aufgaben (Tasks):

- `/identity/tasks`**:** Ruft alle Aufgaben einer Identität ab, inklusive Abhängigkeiten und Metadaten.
- `/tasks`**:** Erstellt neue Aufgaben. Es wird zwischen **dynamischen Tasks**, die Aufwand und Dauer besitzen, und **statischen Tasks**, die blockierend oder nicht blockierend sein können, unterschieden. Aufgaben können zusätzlich Labels, Beschreibungen und Wiederholungsregeln (RRULE nach iCalendar-Standard) enthalten.
- `/tasks/{id}`**:** Ruft einzelne Tasks ab, aktualisiert sie oder löscht sie. Beim Löschen werden auch alle zugehörigen Daten wie Labels, Abhängigkeiten und Unterobjekte entfernt.
- `/tasks/{id}/dependencies`**:** Erstellt eine Abhängigkeit zu einer anderen

Aufgabe, die erfüllt sein muss, bevor die Aufgabe geplant werden kann. Abhängigkeiten können auch wieder gelöscht werden.

Für die automatische Planung von Aufgaben gibt es den Endpunkt:

- `/plan`**:** Startet den Planungsalgorithmus, der Aufgaben anhand von Abhängigkeiten, Zeitfenstern und Arbeits-Slots automatisch einplant. Aufgaben werden topologisch sortiert und in verfügbare Zeitblöcke eingeordnet. Blockierende Tasks können andere Aufgaben verhindern.

Zusätzlich können über `/identity/scopes` alle `Scopes` abgerufen werden.